CHAPTER 5

LINEAR REGRESSION AND GRADIENT DESCENT

NOTE: This chapter is based on 19.6 of Artificial Intelligence a Modern Approach (4th Ed.) by Stuart Russel and Peter Norvig

5.1 Introduction

Linear regression is a supervised learning algorithm in machine learning and statistics. It is used to model the relationship between a dependent variable (Y) and one or more independent variables ($x_1, x_2...$)by fitting a linear equation to observed data.

5.2 Mathematical Model

Linear regression assumes that the output variable y is a linear combination of the input features $\mathbf{x} = [x_1, x_2, \ldots, x_n]$, with an additive error term ϵ. The general form of the linear regression model is:

$$y = w_0 + w_1 x_1 + w_2 x_2 + \cdots + w_n x_n + \epsilon \tag{5.1}$$

Or in vectorized form:

$$y = \mathbf{w}^\top \mathbf{x} + \epsilon \tag{5.2}$$

Here,

- $\mathbf{x} \in \mathbb{R}^n$ is the input feature vector.
- $\mathbf{w} \in \mathbb{R}^n$ is the weight vector (parameters).
- w_0 is the intercept.
- ϵ is the noise or error term.

5.3 Learning Objective

In supervised learning, the core goal is to find a model that makes accurate predictions on unseen data. To achieve this, we define a **loss function**, which measures how well our model's predictions match the actual outputs. In linear regression, a widely used loss function is the **Mean Squared Error (MSE)**.

Mean Squared Error as a Loss Function

For a dataset of m training examples $\{(\mathbf{x}_i, y_i)\}_{i=1}^{m}$, where $\mathbf{x}_i \in \mathbb{R}^n$ and $y_i \in \mathbb{R}$, the model predicts outputs as:

$$\hat{y}_i = \mathbf{w}^\top \mathbf{x}_i \tag{5.3}$$

The Mean Squared Error (MSE) loss function is defined as:

$$\mathcal{L}(\mathbf{w}) = \frac{1}{m} \sum_{i=1}^{m} (y_i - \hat{y}_i)^2 \tag{5.4}$$

Where:

- $\mathcal{L}(\mathbf{w})$ is the loss function quantifying the average squared difference between predicted values and actual values.
- m is the number of training samples.
- y_i is the actual output for the i-th sample.
- $\hat{y}_i$ is the predicted output: $\hat{y}_i = \mathbf{w}^\top \mathbf{x}_i$.

Our objective is to **minimize this loss function** with respect to the weight vector $\mathbf{w}$. Minimizing the loss helps the model generalize well and make accurate predictions.

To find the optimal weights $\mathbf{w}$, we compute the gradient of $\mathcal{L}(\mathbf{w})$ with respect to $\mathbf{w}$ and set it to zero:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{w}} = 0 \tag{5.5}$$

this provides the condition for the minimum.

Solving this system (either analytically or using optimization methods like gradient descent) yields the weight vector $\mathbf{w}$ that minimizes the prediction error across the training data.

5.4 Examples of Linear Regression

Linear regression is widely used in real-world tasks such as:

- Predicting housing prices from features like size and location.
- Estimating a student's exam score based on study hours.
- Modeling sales as a function of advertising budget.

5.5 Univariate Linear Regression

Univariate linear regression, also known as "fitting a straight line", is the simplest case of regression. Here, we consider the output variable (Y) to be dependent of one input variable (x). The output variable takes the form, $Y = w_0 + w_1 x$.

Let us define the weight vector, $W = \langle w_0, w_1$ and define the hypothesis (linear model) with these weights,

$$h_w(x) = w_1 x + w_0 \tag{5.6}$$

can be used to predict the value of the output variable. We are given the following data of students' exam scores based on the number of hours they studied:

Example: Univariate Linear Regression

Hours Studied (x)	Exam Score (y)
1	52
2	55
3	61
4	66
5	71
6	75

Table 5.1: Dataset: Hours Studied vs Exam Score

Let us fit a linear regression model of the form:

$$y = w_0 + w_1 x$$

We can determine the value of the weights solving 5.3 analytically.

Analytical solution

$$\frac{\partial \mathcal{L}}{dw_1} = 0 \text{ and } \frac{\partial \mathcal{L}}{\partial w_0}$$

$$\frac{\partial}{\partial w_0} \sum_{i=1}^{m} (y - (w_1 x + w_0))^2 = 0 \text{ and } \frac{\partial}{\partial w_0} \sum_{i=1}^{m} (y - (w_1 x + w_0))^2 = 0.$$

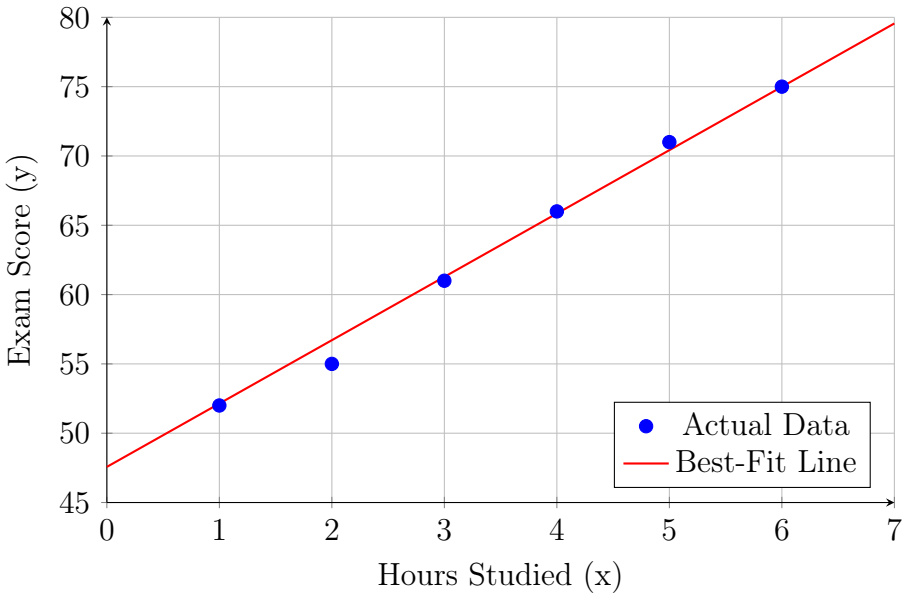
Solving these equations, we get:

$$w_1 = \frac{m\big(\sum x_j y_j\big) - \big(\sum x_j\big)\big(\sum y_j\big)}{m\big(\sum x_j^2\big) - \big(\sum x_j\big)^2}; \qquad w_0 = \frac{1}{m}\Big(\sum y_j - w_1\big(\sum x_j\big)\Big) \tag{5.7}$$

and found that the best-fit line for this data is:

$$\hat{y} = 47.57 + 4.57x.$$

Data Plot with Regression Line



Example: Computing Loss using MSE

Using mean squared error in (5.3) we find the Loss function for the weights, $w_0 = 47.57$ and $w_1 = 4.57$ for the dataset given in table: 5.1.

x_i	y_i	$\hat{y}_i = 47.57 + 4.57x_i$	$y_i - \hat{y}_i$	$(y_i - \hat{y}_i)^2$
1	52	52.14	-0.14	0.0196
2	55	56.71	-1.71	2.9241
3	61	61.28	-0.28	0.0784
4	66	65.85	0.15	0.0225
5	71	70.42	0.58	0.3364
6	75	74.99	0.01	0.0001
Total Sum of Squared Errors				**3.3811**
MSE = Total / 6				**0.5635**

Table 5.2: Step-by-Step Computation of Mean Squared Error (MSE)

5.5.1 Multivariable Linear Regression

In multivariable linear regression, where the output depends on n independent input variables $x_1, x_2, \ldots, x_m$, we aim to find n corresponding weights $w_1, w_2, \ldots, w_m$. These weights are

considered optimal when the partial derivatives of the loss function L with respect to each weight are zero:

$$\frac{\partial \mathcal{L}}{\partial w_1} = 0, \quad \frac{\partial \mathcal{L}}{\partial w_2} = 0, \quad \ldots, \quad \frac{\partial \mathcal{L}}{\partial w_m} = 0 \tag{5.8}$$

Linear regression has a closed-form solution using the normal equation:

$$\mathbf{w} = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{y} \tag{5.9}$$

This solution directly computes the optimal weights that minimize the loss function.

However, computing this closed form solution to find the optimal solution can be complex specially in high dimensional data.

While the analytical solution is faster for small, simple problems, we introduce **gradient descent algorithm** for modern, large-scale, and complex machine learning models due to its flexibility and efficiency.

5.6 Gradient Descent Algorithm

Gradient descent moves iteratively through this space to find the point where the loss is minimized. Starting from an initial point $\mathbf{w}^{(0)}$, we compute the gradient:

$$\nabla \mathcal{L}(\mathbf{w}) = \left[\frac{\partial \mathcal{L}}{\partial w_0}, \frac{\partial \mathcal{L}}{\partial w_1}, \ldots \right] \tag{5.10}$$

We update the parameters in the direction of steepest descent:

$$w_i^{(t+1)} = w_i^{(t)} - \alpha \frac{\partial \mathcal{L}}{\partial w_i} \mid_{w_i = w_i^t} \tag{5.11}$$

Here, α is the learning rate that controls the step size, t represents the iteration number and i refers to the specific parameter in the weight vector being updated.

Algorithm 8 Gradient Descent Algorithm

1: **Input:** Learning rate α, initial weights $\mathbf{w}$, loss function $L(\mathbf{w})$
2: **Output:** Optimized weights $\mathbf{w}$
3: **while** not converged **do**
4: Compute gradient $\frac{\partial \mathcal{L}}{\partial w}$
5: Update weights: $w \leftarrow w - \alpha \frac{\partial \mathcal{L}}{\partial w}$
6: **end while**
7: **return W**

Learning Parameter, α

The parameter α in Equation 5.11 is called the learning rate or step size. It controls how quickly the algorithm updates the weights and how fast it converges to a minimum.

Choosing an appropriate learning rate is important. If α is too large, the algorithm may overshoot or oscillate around the minimum and fail to converge. If it is too small, the algorithm will take a long time to reach the minimum.

The learning rate can be a constant or a decaying parameter. A decaying learning rate helps the algorithm explore more widely in the early stages and fine-tune near the minimum.

Computing Gradient of Loss ($\frac{\partial \mathcal{L}}{\partial w_k}$) in Univariate Linear Regression with Mean Square Error

Let us consider a univariate linear regression with mean square error as the loss function. We already know,

$$\hat{y} = h_w(x) = w_0 + w_1 x.$$

and

$$\mathcal{L} = \frac{1}{m}\sum_{i=1}^{m}(y - \hat{y})^2$$

$$\frac{\partial \mathcal{L}}{\partial w_0} = \frac{\partial}{\partial w_0}\frac{1}{m}\sum_{i=1}^{m}(y - \hat{y})^2$$

Using chain rule of differentiation,

$$\frac{\partial}{\partial w_0}\frac{1}{m}\sum_{i=1}^{m}(y - \hat{y})^2 = \frac{1}{m}\sum_{i=1}^{m}\frac{\partial}{\partial w_0}(y - \hat{y})^2 \tag{5.12}$$

$$= \frac{2}{m}\sum_{i=1}^{m}(y - \hat{y})\frac{\partial}{\partial w_0}(y - \hat{y})$$

$$= -\frac{2}{m}\sum_{i=1}^{m}(y - \hat{y})\frac{\partial \hat{y}}{\partial w_0}$$

$$= -\frac{2}{m}\sum_{i=1}^{m}(y - \hat{y})\frac{\partial}{\partial w_0}(w_0 + w_1 x)$$

$$\frac{\partial \mathcal{L}}{\partial w_0} = -\frac{2}{m}\sum_{i=1}^{m}(y - \hat{y}) \tag{5.13}$$

$$\frac{\partial \mathcal{L}}{\partial w_1} = \frac{\partial}{\partial w_1}\frac{1}{m}\sum_{i=1}^{m}(y-\hat{y})^2$$

Using chain rule of differentiation,

$$\begin{aligned}
\frac{\partial}{\partial w_1}\frac{1}{m}\sum_{i=1}^{m}(y-\hat{y})^2 &= \frac{1}{m}\sum_{i=1}^{m}\frac{\partial}{\partial w_1}(y-\hat{y})^2 \\
&= \frac{2}{m}\sum_{i=1}^{m}(y-\hat{y})\frac{\partial}{\partial w_1}(y-\hat{y}) \\
&= -\frac{2}{m}\sum_{i=1}^{m}(y-\hat{y})\frac{\partial \hat{y}}{\partial w_1} \\
&= -\frac{2}{m}\sum_{i=1}^{m}(y-\hat{y})\frac{\partial}{\partial w_1}(w_0+w_1x) \\
\frac{\partial \mathcal{L}}{\partial w_1} &= -\frac{2}{m}\sum_{i=1}^{m}(y-\hat{y})x_1 \qquad (5.14)
\end{aligned}$$

Computing Gradient of Loss ($\frac{\partial \mathcal{L}}{\partial w_k}$) in Multivariate Linear Regression with Mean Square Error

$$\frac{\partial \mathcal{L}}{\partial w_0} = -\frac{2}{m}\sum_{i=1}^{m}(y-\hat{y}) \qquad (5.15)$$

$$\frac{\partial \mathcal{L}}{\partial w_k} = -\frac{2}{m}\sum_{i=1}^{m}(y-\hat{y})x_k \qquad (5.16)$$

Here, x_k denotes the k'th input variable and w_k is its corresponding weight.

Parameter Update Rule with Gradient Descent

$$w_0^{(t+1)} = w_0^{(t)} + \alpha\frac{1}{m}\sum_{i=1}^{m}(y-\hat{y}). \qquad (5.17)$$

Since, 2 is a constant it can be absorbed into the learning parameter α.

$$w_k^{(t+1)} = w_k^{(t)} + \alpha \frac{1}{m} \sum_{i=1}^{m} (y - \hat{y}) x_k. \tag{5.18}$$

Example: Parameter Update with Loss of Gradient

We compute the gradients of the MSE loss with respect to w_0 and w_1 as follows:

$$\frac{\partial \mathcal{L}}{\partial w_0} = \frac{-2}{m} \sum_{i=1}^{m} (y_i - \hat{y}_i), \quad \frac{\partial \mathcal{L}}{\partial w_1} = \frac{-2}{m} \sum_{i=1}^{m} (y_i - \hat{y}_i) x_i$$

At initial guess $w_0 = 0$, $w_1 = 0$, the prediction $\hat{y}_i = 0$.

Table 5.3: Gradient of Loss with Respect to w_0 and w_1

x_i	y_i	$\hat{y}_i = 0$	$y_i - \hat{y}_i$	$y_i - \hat{y}_i$	$(y_i - \hat{y}_i)x_i$
1	52	0	52	52	52
2	55	0	55	55	110
3	61	0	61	61	183
4	66	0	66	66	264
5	71	0	71	71	355
6	75	0	75	75	450
			Total:	**380**	**1414**

From the table:

$$\sum_{i=1}^{6} (y - \hat{y}) = 380$$

$$\sum_{i=1}^{6} (y - \hat{y}) x_i = 1414$$

Let's take $\alpha = .1$, and with $m = 6$, the after the first iteration the updated weights will be

$$w_0^{(2)} = w_0^{(1)} + \alpha \frac{\partial \mathcal{L}}{\partial w_0}$$

$$= 0 + (.1)\frac{1}{6} \cdot 380$$

and

$$w_1^{(2)} = w_1^{(1)} + \alpha \frac{\partial \mathcal{L}}{\partial w_1}$$

$$= 0 + (.1)\frac{1}{6} \cdot 1414$$

In the second iteration, we will use the updated weights, $w_0^{(2)}$ and $w_1^{(2)}$ to predict $\hat{y}$ and compute the gradient.

The gradient descent algorithm will repeat this process until convergence.

Understanding Gradient Descent in Weight Space

In supervised learning models like linear regression, we represent our model's parameters as a vector $\mathbf{w} = [w_0, w_1, \ldots, w_n]^\top$. Each point in this multi-dimensional space represents a particular configuration of the model.

Weight Parameter Space

Consider a model with two parameters w_0 (intercept) and w_1 (slope). The space formed by all possible values of these parameters is called the **weight parameter space**. Every point (w_0, w_1) in this space corresponds to a different hypothesis or model.

Loss Surface Over Parameter Space

We define a loss function $\mathcal{L}(\mathbf{w})$, such as Mean Squared Error (MSE), that measures how well a model with parameters $\mathbf{w}$ fits the training data. The loss function can be visualized as a surface defined over the weight space. The height of the surface at each point $\mathbf{w}$ corresponds to the loss for that model.

Geometric Intuition

Gradient descent can be viewed as a ball rolling downhill on the loss surface:

- The gradient points in the direction of steepest ascent.
- The negative gradient is the direction of steepest descent.

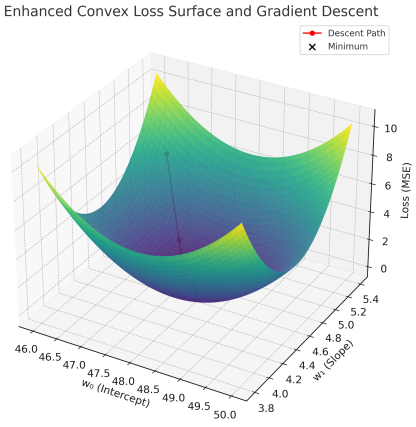


Figure 5.1: Loss vs Weights

- Each iteration moves the parameter vector **w** closer to the minimum of the loss function.

Example: Gradient Descent Algorithm Applied

Let us apply gradient descent algorithm to the example dataset in 5.1 We have plotted the Loss against the parameter space, w_0 and w_1 in fig:5.1.

We can see that it has taken a convex form because of the quadratic nature of the Loss function. We can identify the minima (point where the loss is minimum) at 47.57 and 4.57.

Benefits of This Perspective

Understanding gradient descent in weight space helps to:

- Visualize optimization as navigation across a surface.
- Interpret convergence behavior (e.g., overshooting, slow descent).
- Tune hyperparameters like the learning rate.
- Understand curvature, flat regions, and saddle points.

Variations of Gradient Descent

Gradient Descent has several variations, depending on how much data is used to compute the gradient at each step. The three most common types are:

1. Batch Gradient Descent

In batch gradient descent, the gradient is computed using the entire training dataset:

$$\mathbf{w} \leftarrow \mathbf{w} - \eta \frac{\partial}{\partial \mathbf{w}} \mathcal{L}_{\text{full batch}}(\mathbf{w})$$

Pros: Accurate direction of descent.
Cons: Can be slow and memory-intensive for large datasets.

2. Stochastic Gradient Descent (SGD)

In SGD, the gradient is estimated using only a single training example at each iteration:

$$\mathbf{w} \leftarrow \mathbf{w} - \eta \frac{\partial}{\partial \mathbf{w}} \mathcal{L}_i(\mathbf{w})$$

Pros: Fast and can escape local minima.
Cons: Noisy updates, may not converge smoothly.

3. Mini-batch Gradient Descent

This is a compromise between batch and stochastic gradient descent. The gradient is computed on a small batch of examples (e.g., 32 or 64 samples):

$$\mathbf{w} \leftarrow \mathbf{w} - \eta \frac{\partial}{\partial \mathbf{w}} \mathcal{L}_{\text{mini-batch}}(\mathbf{w})$$

Pros: Efficient and stable. Widely used in practice.
Cons: Requires tuning batch size for best performance.

Why Use Gradient Descent?

Gradient descent is often preferred when:

- The dataset is too large to store or invert the matrix $\mathbf{X}^\top \mathbf{X}$
- The model is nonlinear or does not have a closed-form solution
- We want to train on streaming data in real-time (online learning)
- Regularization techniques are used (e.g., L1, L2)

5.7 Advantages of Linear Regression

- It is **interpretable** and easy to implement.
- It can be solved analytically using closed-form solutions or optimized using gradient descent.
- It forms the basis of more complex models like logistic regression and neural networks.